

2.6 Nested Classes

Java allows a class definition to be *nested* inside the definition of another class. The main use for nesting classes is when defining a class that is strongly affiliated with another class. This can help increase encapsulation and reduce undesired name conflicts. Nested classes are a valuable technique when implementing data structures, as an instance of a nested use can be used to represent a small portion of a larger data structure, or an auxiliary class that helps navigate a primary data structure. We will use nested classes in many implementations within this book.

To demonstrate the mechanics of a nested class, we consider a new Transaction class to support logging of transactions associated with a credit card. That new class definition can be nested within the CreditCard class using a style as follows:

```
public class CreditCard {  
    private static class Transaction { /* details omitted */ }  
  
    // instance variable for a CreditCard  
    Transaction[] history;           // keep log of all transactions for this card  
}
```

The containing class is known as the *outer class*. The *nested class* is formally a member of the outer class, and its fully qualified name is *OuterName.NestedName*. For example, with the above definition the nested class is *CreditCard.Transaction*, although we may refer to it simply as *Transaction* from within the *CreditCard* class.

Much like packages (see Section 1.8), the use of nested classes can help reduce name collisions, as it is perfectly acceptable to have another class named *Transaction* nested within some other class (or as a self-standing class).

A nested class has an independent set of modifiers from the outer class. Visibility modifiers (e.g., **public**, **private**) effect whether the nested class definition is accessible beyond the outer class definition. For example, a **private** nested class can be used by the outer class, but by no other classes.

A nested class can also be designated as either **static** or (by default) nonstatic, with significant consequences. A **static** nested class is most like a traditional class; its instances have no association with any specific instance of the outer class.

A nonstatic nested class is more commonly known as an *inner class* in Java. An instance of an inner class can only be created from within a nonstatic method of the outer class, and that inner instance becomes associated with the outer instance that creates it. Each instance of an inner class implicitly stores a reference to its associated outer instance, accessible from within the inner class methods using the syntax *OuterName.this* (as opposed to **this**, which refers to the inner instance). The inner instance also has private access to all members of its associated outer instance, and can rely on the formal type parameters of the outer class, if generic.